

CmpE 160 Assignment 2

Super Mario Bros. Report

Sude Naz Aslan – 2024400336

01

Spring 2026

1 Introduction

In this assignment, we developed a 2D platform game inspired by the classic Super Mario Bros using Java and the StdDraw graphics library. In this nostalgic game, there are 4 levels with different characteristics. Players should pass through multiple levels avoiding enemies, collecting coins, and utilizing portals to reach exit pipes. We used core object oriented programming topics creating classes for game objects. The gameplay video showcasing all features can be viewed [here](#).

2 Game Dynamics

2.1 Main Menu and Calibration

The game begins with a main menu screen (Figure 1) where players can adjust fps based on their preferences. Mario can move on main screen which enables players to observe different speed variations. Changing FPS actually changes pause duration which can be calculated with (Equation 1):

$$\text{FPS} \approx \frac{1000}{\text{pauseDuration}} \quad (1)$$

Pressing the left arrow key increases the pauseDuration, which decreases FPS and slows down the game. Pressing the right arrow key decreases pauseDuration, increasing FPS and speeding up gameplay. After adjusting FPS, player can start the game by pressing SPACE.

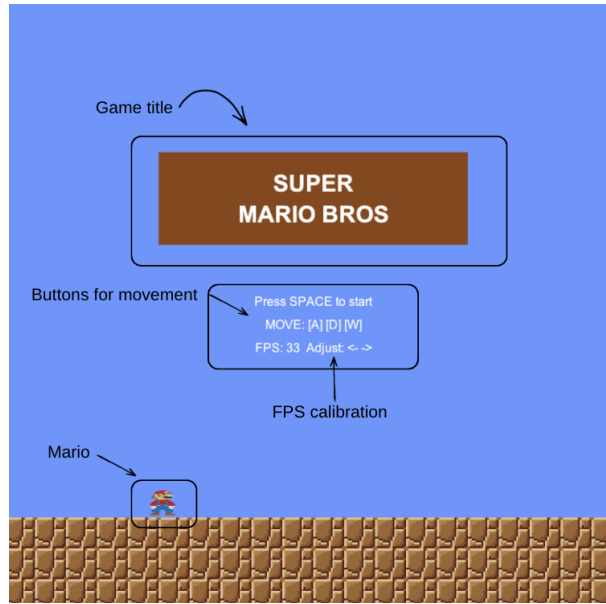


Figure 1: Main menu screen

2.2 Gameplay Mechanics

2.2.1 Level Overview

The game consists of four distinct levels:

1. **Level 1:** Introduction level in which there is no challenge or difficulties. Players can experience the overall structure of the game.
2. **Level 2:** Two enemies move between specific coordinates and Mario needs to reach exit pipe without touching any enemies. (Figure 2)

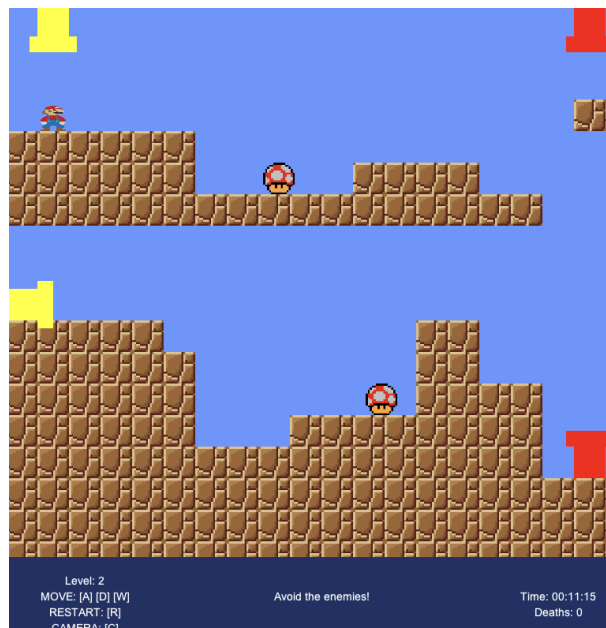


Figure 2: Level 2.

3. **Level 3:** Multiple coins are put throughout the map with moving mushrooms as in level

2. Mario needs to collect all coins and manage to get to the exit pipe without touching the mushrooms. (Figure 3)



Figure 3: Level 3

4. **Level 4:** Custom level with enhanced mechanics or new enemy types. (Described in custom level part in detail.)

2.2.2 Player Movement and Animations

Mario responds to keyboard input using the following controls: A: Move left D: Move right W: Jump S: Crouch (used to enter lower portals) C: Toggle camera mode R: Restart game

To resemble real-life movements, we applied constant gravity on Mario that makes it stand on ground and fall down if jumped by after pressing W. It reduces its Y speed and moves toward ground. Also, double jumping is not allowed which means Mario needs to be on ground before jumping.

Animations: In order to make animations, I used two walking states for each directions between specific frame durations. It switches between them every 5 frames which makes it feel like a real-life situation. Also for jumping, the player has different looks for each direction. When on ground standing still, player turns his body towards directions doing nothing. Finally, for death animation, it initially jumps and after, it stays crouched until it goes beyond the edges of the screen. (Figure 4)



Figure 4: Moment after death animation starts

The following code snippet shows how Mario handles its status for given frame :

```

1  public int handleInput(boolean onGround, int frame) {
2      isWalking = false;
3      int move = 0;
4      if (onGround) {
5          canJump = true;
6      }
7      //handles movements with buttons pressed
8      if (StdDraw.isKeyPressed(KeyEvent.VK_A)) {
9          isWalking = true;
10         move = -5;
11         direction = "left";
12     }
13     if (StdDraw.isKeyPressed(KeyEvent.VK_D)) {
14         isWalking = true;
15
16         move = 5;
17         direction = "right";
18     }
19     if (StdDraw.isKeyPressed(KeyEvent.VK_W) && canJump) {
20         jump();
21     }
22     return move;
23
24 }

```

Listing 1: Mario input handling

2.2.3 Collision Detection System

Collision detection is managed by Map class in the code. Since single collision detection method was not enough for me, I coded checkcollisionAbove and Below helper methods for ceiling and ground state detection. General collision method can be seen in Listing 2. This method is used when checking collision between obstacles, pipes or portals. The main logic of the method follows: After obtaining widths, heights, x and y coordinates of obstacles and mario, it adds or

subtracts height or width of mario and obstacle based on direction of the edge of the objects. For a collision to occur, mario's box must be overlapping with obstacle's box. I added some marginal value in order to make better collision between objects. (Listing 2)

```

1      public boolean checkCollision(double nextX, double nextY, double
      playerHalfSize, int[][] obstacles) {
2      //collision check for single objects
3          for (int[] obs : obstacles) {
4              int object_x = obs[0], object_y = obs[1], object_width =
              obs[2], object_height = obs[3];
5              if (nextX + playerHalfSize - 7 > object_x - object_width
6                  && nextX - playerHalfSize + 7 < object_x +
              object_width
7                  && nextY + playerHalfSize - 7 > object_y -
              object_height
8                  && nextY - playerHalfSize + 7 < object_y +
              object_height) {
9                  return true;
10             }
11         }
12         return false;

```

Listing 2: Collision detection method

For ground and ceiling control, I just checked whether there is collision below or above with obstacles. I used coordinates of corresponding edges of obstacles to compare with Mario's coordinates after adding or subtracting with half of its height.

```

1
2      private boolean checkCollisionBelow(int player_x, int player_y,
      double halfSize, int[] rect) {
3          int rectangle_x = rect[0], rectangle_y = rect[1], rect_width
      = rect[2], rect_height = rect[3];
4          return player_x >= rectangle_x - rect_width - 1 && player_x
      <= rectangle_x + rect_width + 1
5              && player_y - halfSize >= rectangle_y - rect_height
      && player_y - halfSize <= rectangle_y + rect_height;
6      }

```

Listing 3: Collision Below Control

2.2.4 Enemy Mechanics

Enemies moves between two specific x-coordinates x1 and x2. Player needs to pass those areas without touching the enemies. If they hit them mistakenly, death count on HUD increases and death animation starts after which it respawns mario to start pipe to start the game again. (Figure 5)

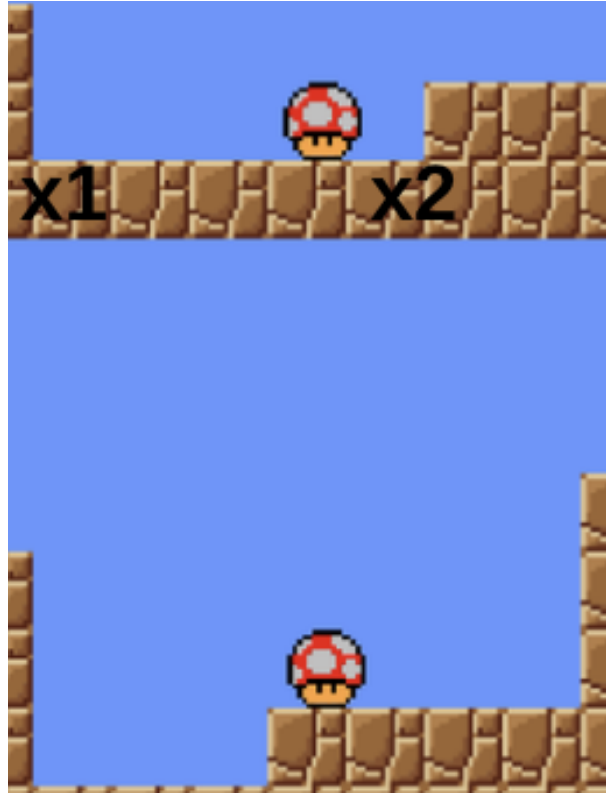


Figure 5: Enemies moving in fixed boundaries

2.2.5 Portal and Teleportation System

Portals allow mario to spawn between distant map points. To use lower portal, player should press S while jumping is enough to enter upper portal. They act as solid objects when they are hit from sides. In code, lower flag is used to control movement between two portals. When it uses lower portal (Figure 6), lower flag becomes active and obstacle collision check is disabled until it reaches the floor passing through upper portal. Mario also acquires extra velocity when moving out from lower portal.

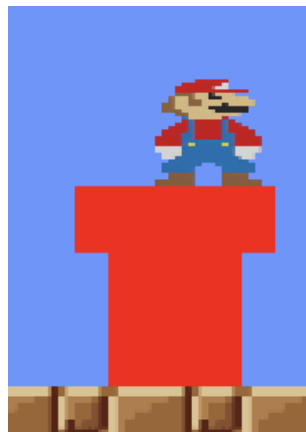


Figure 6: Mario on portal

2.2.6 Camera Mode

Fullscreen Camera Mode: In this mode, camera captures entire screen and players can see full level layout.

Player Following Camera Mode: In this mode, zoomed-in view can be seen on the screen by pressing C.(Figure 7) Camera tracks Mario until map boundaries. I implemented this mode as follows: Firstly in this mode, screen size is 400X400. Thus, since Mario will be at the center of the screen, I assigned coordinates of Mario to camera_x and camera_y variables and set scale according to camera's center positions. Also, when Mario reaches the edges of the screen, camera does not go beyond edges and maintain maximum of width-200, height -200 and minimum 200 x and y coordinates.

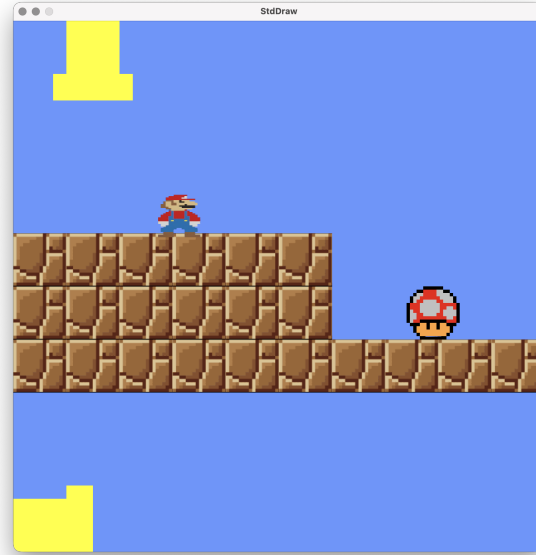


Figure 7: Camera Mode

2.2.7 HUD and Time Display

HUD shows several game information:

- Elapsed time in MM:SS:ms format
- Current level number
- Death counter (total deaths in the game)
- Current level clue
- Keyboard control reminders

2.2.8 Level Completion

To complete a level, player should successfully complete level's unique goal such as avoiding enemies, collecting coins etc. After managing reaching the exit pipe, level banner appears and by pressing SPACE, player can start next level. (Figure 8)

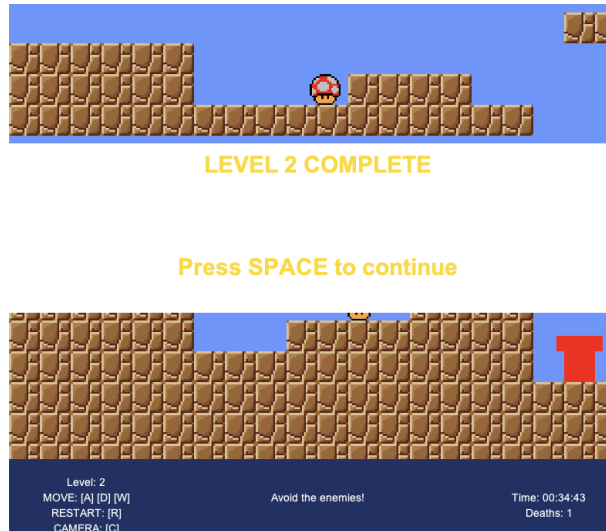


Figure 8: Level completion banner.

2.3 Stage 3: End Game

After successfully completing all four levels, the end game menu is displayed (Figure 9). From the endgame screen, player can choose whether to exit the game or restart again. There can be seen death counts and time elapsed.

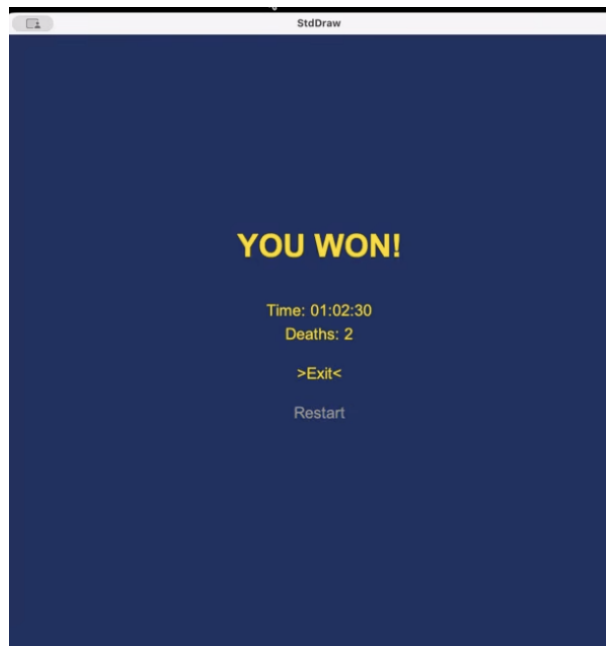


Figure 9: End game menu after all levels are completed.

2.4 Classes

In the game, classes interact with each other in order to implement structured game design. For instance, game class uses map class methods to check whether there is collision with obstacles, to check whether mario is on the ground, touches ceiling or uses mario class to retrieve mario's coordinates and movement. Moreover, main class initializes levels, creates the map and starts the game. All classes has some main functionalities and they come with necessary attributes and methods. UML diagrams can be seen on Figure 10.

Main Class: Initializes the canvas, loads map data from text files, creates the Map object, constructs Level objects, and starts the game.

Game Class: Manages the game loop, physics application, event handling, camera management, and screen transitions. It connects Mario's input intent with Map's collision validation.

Mario Class: Handles input detection, enemy collision, manages movement state, tracks sprite animations, and implements respawn, death logic.

Map Class: Stores static world geometry (obstacles, pipes, portals) and provides collision detection methods, coin collection tracking, portal teleportation logic, and exit detection.

Enemy Class: Manages autonomous movement between predefined boundaries, rendering, and collision box information.

Level Class: Stores level-specific data including ID, enemy list, objective hint text, and coin coordinates. Enables modular level design.

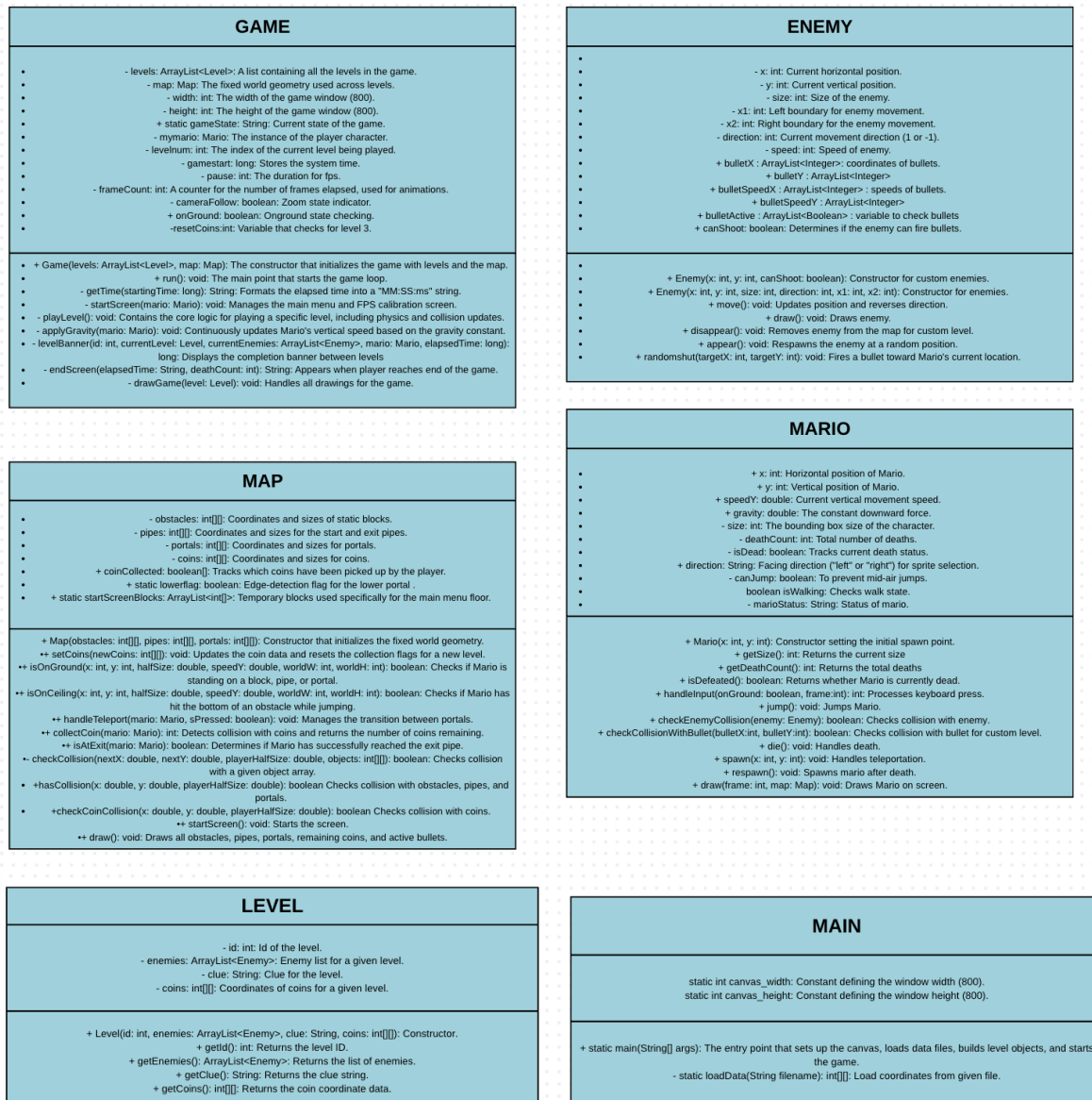


Figure 10: UML class diagram

3 Custom Level

For the custom level, I introduced new enemy type which disappears and then appears randomly based of probability calculated by Math class. Once number created meets the criteria to shoot, enemy fires a bullet towards mario coordinates and disappears. After some seconds, it appears again at one of the coordinates hardcoded in the code. That coordinate is again random. Random movement of enemy is achieved by creating numbers using Math class on Listing 4.

```
1         for (Enemy enemy : currlevel.getEnemies()) {
2             if (enemy.canShoot) {
3                 if (Math.random() < 0.007) {
4                     enemy.randomshut(mymario.x, mymario.y);
5                 }
6             }
7         }
8     }
```

Listing 4: Custom feature implementation

Bullets enemies sent have x and y velocities which are calculated in the code snippet in Listing 5:

```
1
2         double angle = Math.atan2(targety - y, targetx - x);
3         int speedX = (int)(Math.cos(angle) * bulletSpeed);
4         int speedY = (int)(Math.sin(angle) * bulletSpeed);
5
6     }
```

Listing 5: Bullet speed calculation

After speeds are calculated, bullet moves towards Mario and if they collides death animation occurs as it collides with enemies.

In order to pass that level, Mario has to escape from bullets and enemies at the same time to reach exit pipe. After successfully completing fourth level, endscreen occurs and player chooses between restart or exit.

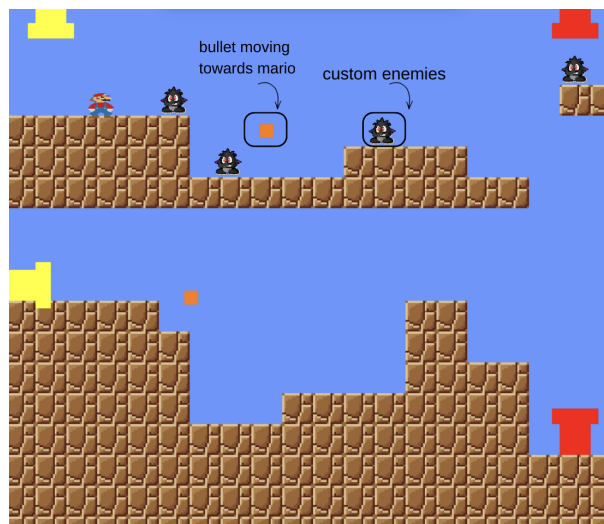


Figure 11: Custom level